

Data Preprocessing, Feature Engineering and Augmentation

Chapter 2: End-to-End Machine Learning Project
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Some Difficulties with Available Data

- When you are learning about Machine Learning it is best to actually experiment with real-world data, not just artificial datasets.
- Fortunately, there are thousands of open datasets to choose from, ranging across all sorts of domains.
- Here are a few places you can look to get data.
- **Popular open data repositories:**
 - Kaggle datasets
 - UC Irvine Machine Learning Repository
 - Amazon's AWS datasets
- **Meta portals (they list open data repositories):**
 - <http://dataportals.org/>
 - <http://opendatamonitor.eu/>
 - <http://quandl.com/>
- In some applications, the training set is created manually, where an expert collects and prepares examples, selects relevant attributes, assigns class labels, and specifies attribute values. In other domains, the process is automated or semi-automated, with attribute vectors extracted from one or more databases and labels assigned either programmatically or by experts. Often, data comes from multiple sources, combining automated database records with manually added examples. Regardless of how the data is collected, it is almost always affected by imperfections, whose nature and impact must be carefully understood by the engineer.

General Steps to Prepare Data before Feeding It to an Algorithm

1. Data Preprocessing and Feature Engineering

- Data is often collected from multiple sources and may be noisy, incomplete, or unreliable. Therefore, it is unrealistic to expect perfectly clean data.
- **Data preprocessing refers to transforming raw data into a clean and usable format** (handling missing values, encoding categorical variables, normalization, etc.).
- **Feature engineering is the process of creating, transforming, or selecting features** that improve model performance.
- The quality of data and the usefulness of extracted features directly affect a model's ability to learn.
- Data preparation is the first and most crucial step in building a machine learning model.
- **It is a labor-intensive process;** in practice, more than half of the project time is often spent dealing with data quality issues.

2. Data Splitting

- The dataset is divided into training, validation, and test sets to ensure unbiased model evaluation.

3. Data Augmentation

- Data augmentation **creates realistic variations of existing data** while preserving feature meaning and relationships.
- Data augmentation is needed:
 - **To expand minority classes in imbalanced datasets**
 - **To improve model generalization and reduce overfitting**
 - **To increase effective dataset size when data collection is expensive**
- Synthetic samples must obey domain constraints.
- Data augmentation is typically applied after handling missing values and preprocessing.
- **Always split the dataset before augmentation.**
The test set must represent unseen, real-world data.
If augmentation is performed before splitting, synthetic samples derived from a single record may appear in both training and test sets, leading to **data leakage** and overly optimistic performance estimates.

Tabular / Numeric Data

Problems with Attributes / Features

- Irrelevant attributes / features
 - Add to computational costs without adding value to the model.
- Missing attributes / features
 - Result in a lack of essential information necessary for accurate predictions.
- Redundant attributes / features
 - Can create confusion, as their values can be derived from other features; may lead to biased output and increased computational cost.
- Missing attribute / feature values
- Inconsistent attribute/ feature values
- Duplicate attribute/ feature values

Noise in a Dataset

- Noise in a dataset refers to random errors or variances in measured values that do not reflect the true underlying data.
- It can arise from various sources and can negatively impact the performance of machine learning models.
- Addressing noise is crucial for enhancing data quality and improving the reliability of machine learning models.
- Both attributes and targets/label can be noisy.

Sources of Noise

- Measurement Error: Inaccuracies during data collection due to instrument flaws.
- Human Errors: Data entry mistakes made during data input can introduce inaccuracies.

Types of Noise

- Random Noise: Arises randomly and is unpredictable, making it difficult to model.
- Systematic Noise: Consistent errors that can be identified and possibly corrected.

Data Preprocessing and Feature Engineering

Handling Missing Values

- Two ways to handle missing values are:
 - Eliminate rows with missing data.
 - Simple and sometimes effective strategy.
 - Removing data may lead to loss of information which will not give the accurate output.
 - Estimate missing values:
 - Imputation Technique
 - Fill the missing values with the mean, median or mode value of the respective feature.
 - Interpolation Technique
 - Interpolation is used to estimate missing values based on neighboring data points, assuming that the data follows a certain trend or continuity.
 - Unlike mean or median imputation, interpolation uses the order or structure of the data, making it especially useful for time-series or sequential data.
 - Common interpolation methods include:
 - **Building a simple model** (e.g., linear or polynomial) using known data points, which is then used to estimate missing values.
 - **Forward Fill (Last Observation Carried Forward)**, where the previous known value is used.
 - **Backward Fill (Next Observation Carried Backward)**, where the next known value is used.
 - Interpolation should not be used for unordered tabular data where rows are independent, as inappropriate interpolation may introduce false patterns or trends into the data.
 - Mean/Median Imputation ignores data order, whereas Interpolation exploits data continuity.
- Both the techniques work if only a reasonable percentage of values are missing.
- If a feature has mostly missing values, then that feature itself can also be eliminated.

Handling Inconsistent Values

- Data may contain inconsistent values.
- It may be due to human error or maybe the information was misread while being scanned from a handwritten form.
- It is therefore always advised to perform data assessment like knowing what the data type of the features should be and whether it is the same for all the data objects.

Handling Duplicate values

- A dataset may include data objects which are duplicates of one another. It may happen when say the same person submits a form more than once.
- In most cases, the duplicates are removed so as to not give that particular data object an advantage or bias, when running machine learning algorithms.

Handling Noise

- The simplest way to handle noisy data is to collect more data. The more data you collect, the better will you be able to identify the underlying phenomenon that is generating the data. This will eventually help in reducing the effect of noise.

Categorical Variable

- It is a discrete variable that captures qualitative outcomes by placing observations into fixed groups (or levels).
- The groups are mutually exclusive, which means that each individual fits into only one category.

Types of categorical variables

- **Ordinal data** - represents outcomes for which the order of the groups is meaningful.
- **Nominal data** - represent outcomes for which the order of groups does not matter.
- **Binary data** - data with only two possible outcomes.

Encoding Categorical Variables

- Since machine learning algorithms operate on numerical data, categorical variables must be converted into numeric form.
 - Common approaches include dummy variable encoding, ordinal encoding, and one-hot encoding.

Dummy Variable Encoding

- A dummy variable is a variable that takes values of 0 and 1, indicating the presence or absence of something.
- It is a natural encoding for binary variables.

Ordinal Encoding

- Each unique category is assigned an integer value, reflecting a meaningful order, which some machine learning algorithms can exploit.
- Also called integer encoding; often, integer values starting at zero are used.
- It is a natural encoding for ordinal variables.

One-Hot Encoding

- When no ordinal relationship exists, ordinal encoding may be misleading. Forcing an artificial order may cause models to assume relationships that do not exist, leading to poor performance.
- In one-hot encoding, the original categorical variable is replaced by multiple binary variables, one for each category.
- It is a generalization of dummy variables.
- It is a natural encoding for nominal variables.

Binning

- An opposite situation (to categorical data), occurring less frequently in practice, is to have a numerical feature which is to be converted into a categorical one.
- Binning (also called bucketing) is the process of converting a continuous feature into multiple binary features called bins or buckets, typically based on value range.

Feature Scaling

- Makes sure that the features involved in the computation are on the similar scale (take similar ranges of values).
- It prevents any one feature having high range values, to dominate the results.
- It helps model converge faster and gives more balanced weights to each feature during the learning process, leading to better prediction performance / accuracy.
- Machine learning algorithms, where feature scaling is advantageous include:
 - linear regression, logistic regression, neural network, etc. that use gradient descent optimization,
 - Distance algorithms like K nearest neighbors (KNN), K-means clustering, and support vector machine (SVM).
- Tree-based algorithms like decision trees, random forest, xgboost, etc., on the other hand, are fairly insensitive to the scale of the features.
- Note that scaling the target values is generally not required.
- Two common techniques are Normalization and Standardization.

Normalization

- Also known as min-max scaling or min-max normalization.
- Scales down the values of the features between 0 and 1.
- The general formula for normalization is given as: $x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$
 - x_{max} and x_{min} are the maximum and the minimum values of the feature respectively.
 - When the value of x is the minimum value in the column, the numerator will be 0, and hence x_{norm} is 0.
 - On the other hand, when the value of x is the maximum value in the column, the numerator is equal to the denominator and thus the value of x_{norm} is 1.
 - If the value of x is between the minimum and the maximum value, then the value of x_{norm} is between 0 and 1.

Standardization

- Also called z-score normalization.
- Transforms the feature into standard normal distribution, having zero mean and unit variance.
- The general method of calculation is to determine the distribution mean μ and standard deviation σ for each feature and calculate the new data point by the following formula: $x_{stand} = \frac{x - \mu}{\sigma}$.

where, $\mu = \frac{\sum_1^n x_j^{(i)}}{n}$ (averaging all the values in a column/feature) and $\sigma = \sqrt{\frac{\sum_1^n (x_j^{(i)} - \mu)^2}{n}}$.

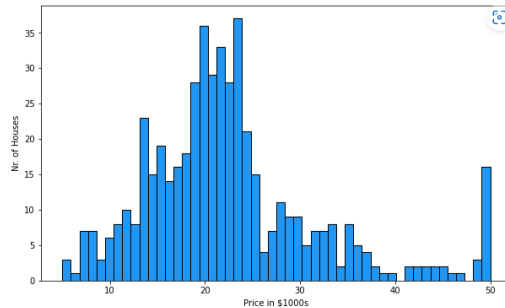
- Here the values are not restricted to a particular range.

Normalization or Standardization?

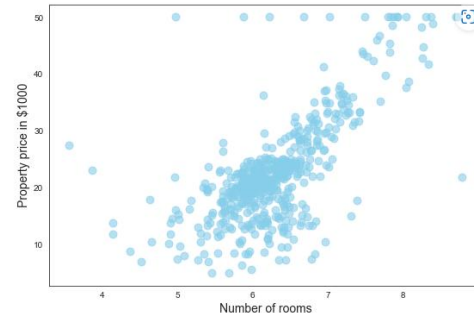
- Standardization can be helpful in cases where the data follows a Gaussian/Normal distribution (the bell-curve).
- Since standardization does not have a bounding range, it works well with outliers in the data.
- Impact of Outliers is very high in Normalization, which will squeeze the normal values into a very small range.
- Unsupervised learning algorithms, in practice, more often benefit from standardization than from normalization.
- For all other cases normalization can be used.
- Normalization is specifically preferred in image processing, where pixel intensities have to be normalized to fit within a certain range (i.e., 0 to 255 for the RGB color range).
- However, there is no hard and fast rule, the choice of using normalization or standardization will depend on the problem and the machine learning algorithm being used.
- You can always start by fitting your model to raw, normalized and standardized data and compare the performance for best results.

Data Visualization

- Data visualization can help gain better insights.
- Scatter plots and histograms are a very useful tool in this regard.
- These plots can be used to spot outliers in the features.
- Histograms help discover data distributions for the features.
- Scatter plots can be plotted between various features or between a feature and the target to discover the corresponding correlations.



Histogram



Scatter Plot

Correlation

- Correlation is a statistical measure that expresses the extent to which two variables are linearly related.
- Correlation is the degree to which things move together.
- The correlation coefficient quantifies the strength of the relationship.
 - It ranges from -1.0 to +1.0
 - A positive value indicates positive correlation.
 - A negative value indicates negative correlation.
 - A zero indicates no correlation.

Formula:
$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

Where,

r = correlation coefficient

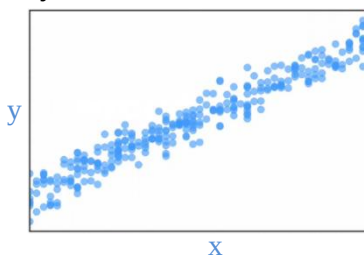
x_i = values of the x variable in the sample

$\bar{x}$ = mean value of the values of the x variable

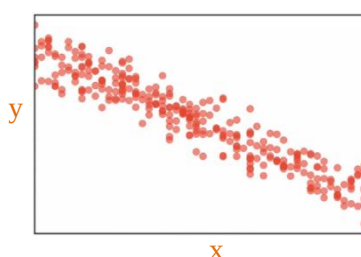
y_i = values of the y variable in the sample

$\bar{y}$ = mean value of the values of the y variable

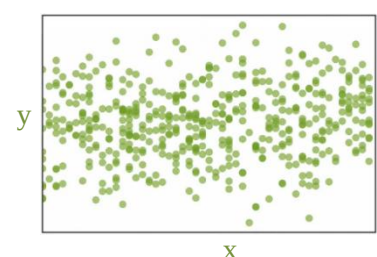
- This type of correlation is called Standard correlation coefficient or Pearson's coefficient.
- The correlation coefficient only measures linear correlations ("if x goes up, then y generally goes up/down"). It may completely miss out on nonlinear relationships (e.g., "if x is close to zero then y generally goes up").
- Graphically



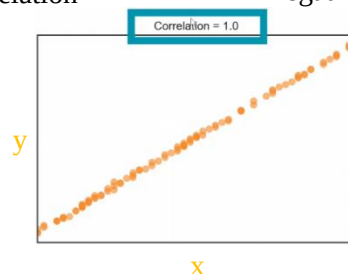
Positive correlation



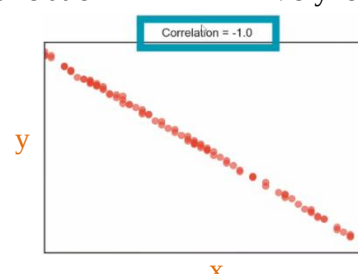
Negative correlation



Very low correlation



Strong positive correlation



Strong negative correlation

Multicollinearity

- High correlation *may* cause multicollinearity.
- Multicollinearity is a phenomenon in which one predictor variable in a multiple regression model can be linearly predicted from the other with a substantial degree of accuracy.
- When two features are highly correlated:
 - each do not provide unique and independent information in regression,
 - leads to unreliable estimates.
- Hence, one of the correlated feature should be removed.

Dimensionality Reduction

- The number of attributes, **features** or input variables of a dataset is referred to as its **dimensionality**.
- Dimensionality reduction simply refers to the process of reducing the number of attributes in a dataset while keeping as much of the variation in the original dataset as possible.
- More input features often make a predictive modeling task more challenging, referred to as the curse of dimensionality.
- **A lower number of dimensions in data means less training time and less computational resources and increases the overall performance of machine learning algorithms.**
- **Also, high dimensional data is likely to overfit the training dataset and therefore may not perform well on new data.**
- It is also extremely useful for data visualization.
- **Dimensionality reduction takes care of multicollinearity.**
- Two common approaches are feature selection and feature projection.

Feature Selection Methods

- Try to find a subset of the original set of variables, or features, to get a smaller subset which can be used to model the problem.
- Two main classes of feature selection techniques include **wrapper methods** and **filter methods**.
- **Wrapper methods**, wrap a machine learning model, fitting and evaluating the model with different subsets of input features and selecting the subset the results in the best model performance. **Multiple features can also be combined into one if possible, called feature aggregation.**
- **Filter methods** use **scoring methods**, like **correlation between the feature and the target variable**, to select a subset of input features that are most predictive.
- Calculating p-values is another statistical tool to check for significance of a feature. **If p-value of a feature is <0.05 then it is significant, otherwise not.**

Feature Projection or Extraction Methods **done via automated tools**

- This reduces the data in a high dimensional space to a lower dimension space.
- It can be done through various methods:
 - **Matrix factorization techniques** from linear algebra, for e.g. principal components analysis (PCA).
 - **Manifold learning techniques from high-dimensionality statistics**, for e.g. Multidimensional Scaling (MDS), t-distributed Stochastic Neighbor Embedding (t-SNE).
 - Autoencoder methods which use deep learning neural networks.

Common Tabular Data Augmentation Techniques

- Tabular data augmentation is challenging because tabular data often contains mixed data types, strong feature dependencies, and imbalanced classes, making careless augmentation risky.

Adding Noise (Jittering)

- **Small random noise is added to numerical features** to create slightly different samples.

Resampling Smaller Classes **make the data balance as well**

- Resampling in categorical feature augmentation can select the same sample multiple times to generate additional training data, especially for balancing classes.

SMOTE (Synthetic Minority Over-sampling Technique)

- Generates synthetic samples for the minority class by interpolating between similar observations.
- Similar observations are identified using k-nearest neighbors (k-NN) in feature space.
- For two minority-class points x_i and x_j , $x_{new} = x_i + r(x_j - x_i)$ where:
 - r is a random number between 0 and 1.
 - x_{new} lies on the line segment connecting x_i and x_j .

Gaussian Copula–Based Synthesis

- Gaussian copula creates a model of the joint distribution (the “data cloud”) and samples new points from it, preserving both individual feature distributions and inter-feature dependencies.

CTGAN (Conditional Tabular Generative Adversarial Network)

- Learns the joint distribution of all features and generates realistic synthetic records.

Variational Autoencoders (VAE)

- Learns a compressed representation of tabular data and samples new points from it.

Python Libraries for Tabular Data Augmentation

- SMOTE, ADASYN, Borderline-SMOTE for handling imbalanced datasets.
- SDV (Synthetic Data Vault), supports CTGAN, TVAE, and Gaussian Copula.
- AugLy, a Meta’s augmentation library, supports tabular data transformations.
- Timesynth for Time-series data augmentation.

Some Commonly Used Tabular Datasets

Dataset	Features	Target	Application / Notes
Iris https://archive.ics.uci.edu/dataset/53/iris https://www.kaggle.com/datasets/uciml/iris?utm	Sepal/petal length and width	Species (Setosa, Versicolor, Virginica)	Multi-class classification, small & clean
Heart Disease (UCI) https://archive.ics.uci.edu/dataset/45/heart?utm	Age, sex, BP, cholesterol, max heart rate, etc.	Heart disease (0/1)	Binary classification, healthcare
Boston Housing	CRIM, RM, AGE, TAX, etc.	Median house price	Classic regression benchmark
California Housing https://inria.github.io/scikit-learn-mooc/python_scripts/datasets_california_housing.html?utm	Numeric housing attributes (median income, house age, rooms, etc.)	Median house value	Larger regression dataset, modern usage
Adult / Census Income https://www.kaggle.com/datasets/ranajawadriaz/uci-adult-income-dataset?utm	Age, Education, Occupation, Hours per week, etc.	Income >50k (0/1)	Binary classification, can illustrate class imbalance
Yeast (UCI)	Protein attributes (sequence, composition, physiochemical features)	Subcellular localization classes	Multi-class / multi-label biological problem